

Okay

- La ejecución final de un **Hola Mundo** (SOLO BACKEND) que abarque:
 - conexión a la BD,
 - instanciación mínima de entidades (como representar con una clase, una entidad de BD y utilizarla)

Para nuestro tutorial enseñamos como montar el back de una aplicación (que es el mismo que usaremos para el proyecto lol) Justificación de Gemini;

"Para nuestro proyecto, justificamos que este es un 'Hola Mundo de Backend' porque va más allá de imprimir texto en una consola. Un verdadero Hola Mundo en el desarrollo web moderno consiste en validar que todo el flujo de datos está vivo y conectado. Al lograr la conexión a la BD , traducir una tabla a código mediante la instanciación mínima de entidades (usando la clase `User` o `Hardware`) , y prepararlo todo para recibir una prueba desde Postman, estamos demostrando que las tres piezas fundamentales de nuestro motor (Base de Datos, ORM/Backend, y Cliente/API) se comunican perfectamente desde cero."

para correr lo que ya tenemos programado:

Ejecutar primero

Deben tener Docker instalado (Se puede desde la microsoft store o desde internet para distros de Linux)

deben abrirlo y que en la esquina inferior diga "Engine Running"

despues de eso van hasta la carpeta exacta donde se encuentra el archivo que hizo Nael docker-compose.yml

docker-compose.yml levanta una base de datos PostgreSQL en un contenedor, la configura con las variables del .env y la primera vez que arranca ejecuta el init.sql para crear todas las tablas automáticamente.

enciendan la bd usando:

sudo docker-compose --env-file ../.env up -d → Esa sintaxis porque el .yml está en una subdirectorio del directorio donde está el .env
(Estoy usando ubuntu, busquen el equivalente de win)

Esta screenshot la pide el profe y dijo que daba puntos extra :p plz como ustedes son los últimos en trabajar las cosas, dejen todo lo que el profe pide en su sitio. pueden volver a ejecutar el comando `docker ps -a` y les debe aparecer lo mismo

```

estudiante@monOrd:~/ingesoft-tutorial/project-backend/scripts$ sudo docker-compose --env-file ../.env up -d
[sudo] password for estudiante:
Sorry, try again.
[sudo] password for estudiante:
WARN[0000] /home/estudiante/ingesoft-tutorial/project-backend/scripts/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please
remove it to avoid potential confusion
[*] up 20/20
✓ Image postgres:16 Pulled 13.6s
✓ Network scripts_default Created 0.1s
✓ Volume scripts_postgres_data Created 0.0s
✓ Container benchmark_db Started 1.1s
estudiante@monOrd:~/ingesoft-tutorial/project-backend/scripts$ docker ps -a
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
estudiante@monOrd:~/ingesoft-tutorial/project-backend/scripts$ sudo docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
2e332b5490b8   postgres:16 "docker-entrypoint.s..." 40 seconds ago Up 39 seconds 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp benchmark_db
estudiante@monOrd:~/ingesoft-tutorial/project-backend/scripts$

```

Nota:

Las librerías psycopg2-binary y python-dotenv:

psycopg2-bin permite a Python conectarse y comunicarse con bases de datos PostgreSQL. Sin esto, Django no puede ejecutar consultas SQL en una base de datos PostgreSQL. Lo uso en settings

Chequen el archivo models.py. Allí hay información importante sobre el enlace de Django a la bd. Les recomiendo preguntarle a Gemini que otros cambios chiquitos hice por ahí de los cuales no me acuerdo pero están en los archivos de settings de Django.

La base de datos no está poblada aún, entonces porfa para el test, poblen un registro de la tabla Videogame usando la consola de Django. Después de eso si pueden intentar sacar los datos de Postman

Voy a subir todo esto como un branch llamado Tutorial porque la idea es que todos los archivos que voy a subir no estén en sí en el repositorio.

Para David:

Guía de Configuración del Backend (Rama Tutorial)

¡Hola! Para que no tengas problemas configurando el proyecto, te preparé esta guía paso a paso. Síguela en orden y tendrás todo corriendo en un par de minutos.

1. Descargar el código

Abre tu terminal y clona el repositorio. Luego, pásate a la rama **Tutorial**, que es donde están todas las configuraciones listas:

Bash

```

None
git clone
https://github.com/Centro-Democratico/project-backend.git
cd project-backend
git fetch origin

```

```
git checkout Tutorial
```

2. Configurar las variables de entorno

Necesitamos crear tu archivo `.env` a partir de la plantilla. **Si usas Windows (CMD):**

```
None  
copy .env.example .env
```

Si usas Linux/Mac/WSL:

```
None  
cp .env.example .env
```

3. Levantar la Base de Datos con Docker

Asegúrate de tener Docker Desktop abierto y corriendo. **En Windows:**

DOS

```
None  
cd scripts  
docker-compose --env-file ../.env up -d  
cd ..
```

En Linux/Mac/WSL:

Bash

```
None  
cd scripts  
sudo docker-compose --env-file ../.env up -d  
cd ..
```

4. Preparar el entorno de Python

Vamos a instalar las librerías necesarias (Django, PostgreSQL, etc.): **En Windows:**

DOS

None

```
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt
```

(si algo falla ejecute

None

```
pip install djangoRESTframework
```

, si sigue fallando no tengo ni idea)

y de ahí pues, cree el dato de la tabla de videogame y testee todo lo que haya que testear.

Sus tareas son:

poblar la base de datos con al menos un dato hacer el test con postman, crear el .sh y el .bat

<https://docs.google.com/document/d/19-NQmZwZQX6whx4Mem0zWrOapuJkkWSiBeEPICdlykg/edit?usp=sharing>